UnrealChrono

Architecture, the Chrono integration, the Chaos collision backend, physical materials and collision channels, and the Bullet vs Chaos comparison results

Unreal Engine 5.8 · Project Chrono 10.0.0 · generated 2026-10-07 from the project sources and the recorded test runs

What this document covers

- **How the project is built:** the game module, the ChronoCore and ChronoVehicle plugins, and which thread owns what.
- **How it talks to Chrono:** the whole `chrono::ChSystem` runs inside a Chaos sim callback; only plain data crosses between the game thread and the physics thread.
- **How the Chaos backend works:** Chrono bodies are mirrored as kinematic proxy particles in Unreal's solver, so they collide with each other and with the level.
- **How physical materials and collision channels decide the result** for both backends (Chaos and Bullet).
- **What the Bullet vs Chaos comparison test measured:** method, results, what agrees and what does not, and the bug it found.

Contents

1 Overview 2

2 Modules, plugins and unit conventions 4

3 One physics tick 5

4 The Chrono vehicle model 7

5 The Chaos collision backend 9

6 Physical materials 12

7 Collision channels and collision families 14

8 Terrain deformation 16

9 Backend comparison test: method 17

10 Results 18

1 Overview

UnrealChrono is a minimal Unreal Engine project that runs Project Chrono vehicle dynamics with a primitive test car on a landscape that deforms under its wheels. Chrono is compiled from source into two engine plugins. The vehicle, its driver, its tires and its terrain sensing live in a `chrono::ChSystem` that is stepped on Unreal's Chaos physics thread, decoupled from the frame rate. The Unreal side only sends driver inputs and one-shot commands in and reads immutable snapshots out.

Collision detection is pluggable. The Chaos backend delegates broadphase, narrowphase and ray queries to Unreal's own physics scene, so Chrono bodies see the level directly (landscape, static meshes). The Bullet backend uses Chrono's bundled Bullet and sees only Chrono bodies, so the level geometry is exported to it.

Main classes

Class	Thread	Role
ACarPhysicsWorld	game	hosts the FChronoSimulation, builds the FChronoVehicleSetup from the actors, pushes inputs, pumps snapshots, selects Chaos or Bullet, exports level geometry for Bullet
APhysicsVehicle	game	pawn made of engine primitives; polls the keys, applies the snapshot to the meshes, sends one terrain-deformation stamp per wheel
FChronoSimulation (ChronoCore)	game facade	registers the sim callback on the solver of the world's physics scene; Enqueue / PushInputs / PumpOutputs / GetLatestSnapshot
FChronoSimCallback (ChronoCore)	physics	owns the ChSystem; per physics step: inputs, commands, fixed substeps, snapshot
FChronoVehicleWorld	physics	IChronoStepper: builds TestCar, Driver and terrain; per substep synchronizes and advances them and steps the system; builds the FVehicleSnapshot
TestCar : ChronoVehicle	physics	the Chrono wheeled vehicle (see section 4)
ChaosTerrain / ChronoTerrain	physics	terrain queries for the tires: Chaos ray casts against the Unreal scene / Bullet ray casts against the exported colliders
FUnrealCollisionExtractor, FStaticCollisionGrid, FBulletStaticCollision	game + worker	Bullet only: turn tagged Unreal geometry into static Chrono bodies, stream them in and out around the vehicle
UTerrainDeformSubsystem, FTerrainDeformViewExtension, FTerrainDeformPhysicsCache	game / render	GPU height-delta deformation of the Nanite landscape and its berm feedback to the physics
AChronoBackendComparison	game	the Bullet vs Chaos comparison test (sections 9 and 10)

Threading contract

- The `ChSystem` is touched only on the physics thread; the game thread never blocks on it. Commands, the system configuration hook and everything inside the system must not touch UObjects or Unreal scene APIs.
- **Inputs are states** ("throttle is 0.6"), double buffered and possibly handed to several physics steps or none. **Events** such as gear shifts and the vehicle build are one-shot commands.
- **Outputs are immutable snapshots** (shared pointers) built on the physics thread, so the game thread reads them without locks. The newest one wins.
- Anything the physics thread needs from UObjects is captured on the game thread first (for example the physical material table, see section 6).

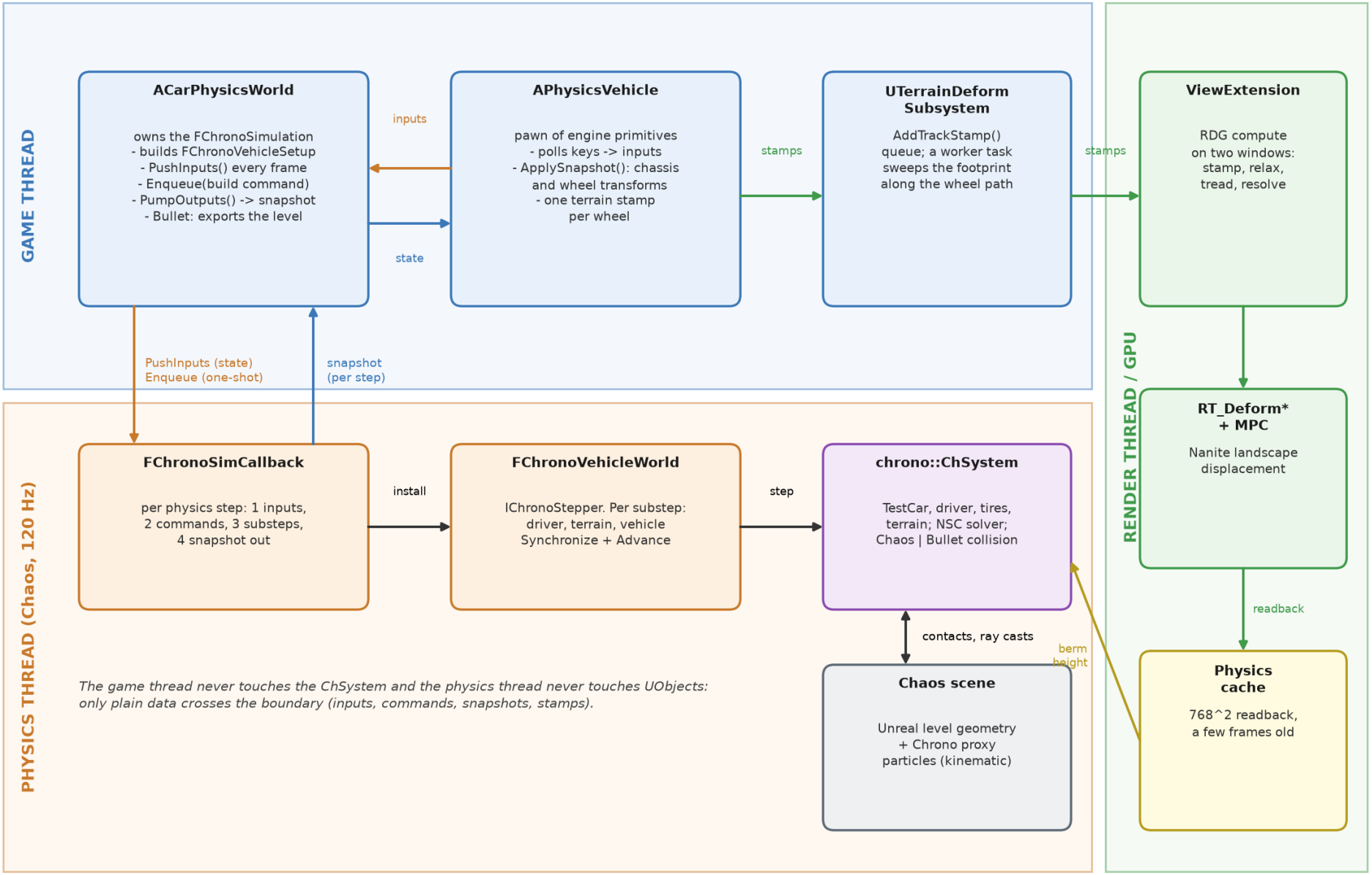
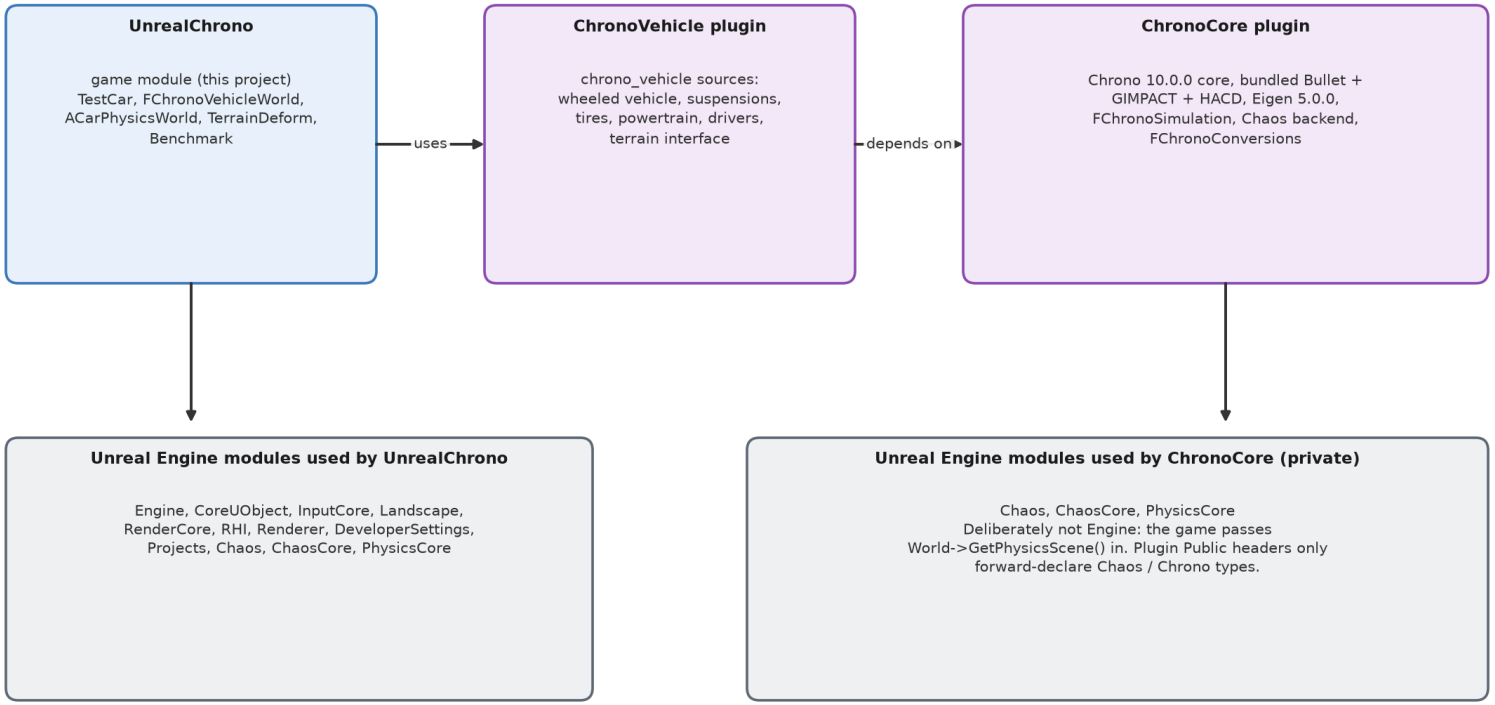


Figure 1: runtime architecture. Blue = game thread, orange = physics thread, purple = Chrono, green = render thread / GPU, yellow = data shared between threads.

2 Modules, plugins and unit conventions



Chrono headers are never included next to Unreal / Windows headers directly: ChronoIncludeBegin.h / ChronoIncludeEnd.h bracket them (THIRD_PARTY_INCLUDES, NOMINMAX, check/verify/TEXT macros, MSVC warnings). Chrono needs RTTI and C++ exceptions in every consumer module.

Figure 2: module dependencies.

	Chrono	Unreal
Length unit	metres	centimetres
Handedness	right handed, Z up	left handed, Z up
Position	(x, y, z) m	(x, −y, z) × 100 cm: the Y axis is mirrored and scaled
Quaternion	(w, x, y, z)	(−w, x, −y, z) of the Chrono quaternion (FChronoConversions)
Gravity	(0, 0, −9.81) m/s²	(0, 0, −981) cm/s² (FChronoSimulationParams::GravityUE)

3 One physics tick

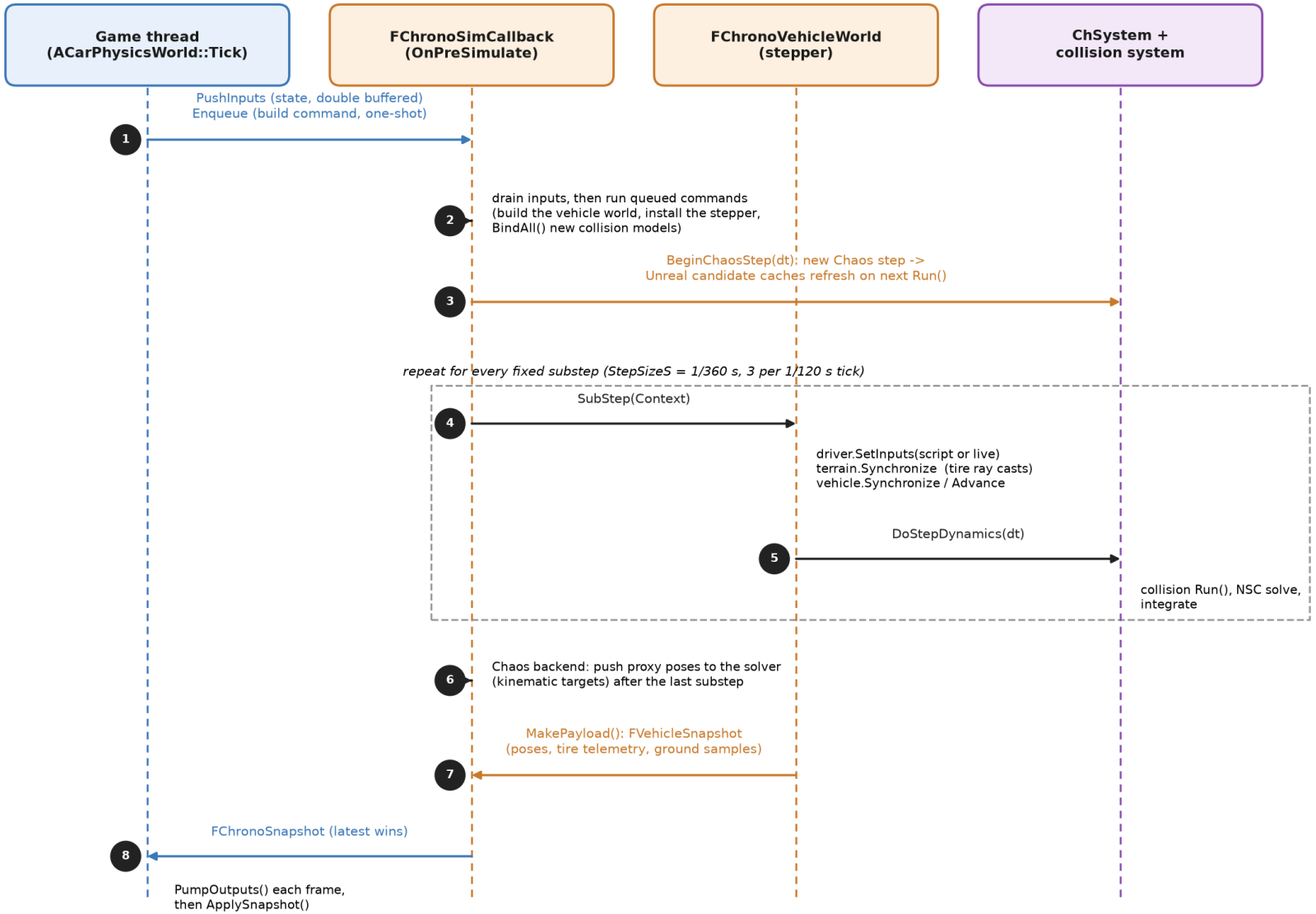


Figure 3: sequence of one Chaos physics step. Steps 4 and 5 repeat for every fixed Chrono substep.

3.1 Timing

- Project Settings > Physics > **Tick Physics Async** is on with `AsyncFixedTimeStepSize = 1/120 s`: the physics thread ticks at a fixed rate independent of the frame rate.
- Chrono substeps: `StepSizeS = AsyncFixedTimeStepSize / ChronoSubStepsPerPhysicsTick` = 1/360 s (3 substeps per tick, 2.78 ms). Without async physics the step is 1/360 s and at most 32 substeps run per tick; any backlog beyond the cap is dropped.
- Contacts are Chrono NSC (complementarity) with the iterative PSOR solver, 140 iterations.
- Chaos-specific hooks around the substeps: `BeginChaosStep` (marks a new Chaos step so the Unreal candidate caches refresh once per step) and `EndChaosStep` (pushes the proxy poses to the solver as kinematic targets).

3.2 What the stepper does in each substep

- **Driver:** inputs come from the game thread (live play) or from a script indexed by the time since the vehicle was built (comparison test), so the result is independent of the frame rate.
- **Terrain.Synchronize**, then **Vehicle.Synchronize**: the tires ask the terrain for height, normal, friction and surface type (a ray cast through the chosen backend).
- **Advance** driver, terrain and vehicle, then `ChSystem::DoStepDynamics(dt)` (the vehicle does not own the system, so it never steps it itself).
- After the substeps `MakePayload` builds the `FVehicleSnapshot`: chassis and spindle transforms, per-tire telemetry (slip, deflection, contact, terrain type and friction) and the ground height samples of the discrete tire.

4 The Chrono vehicle model

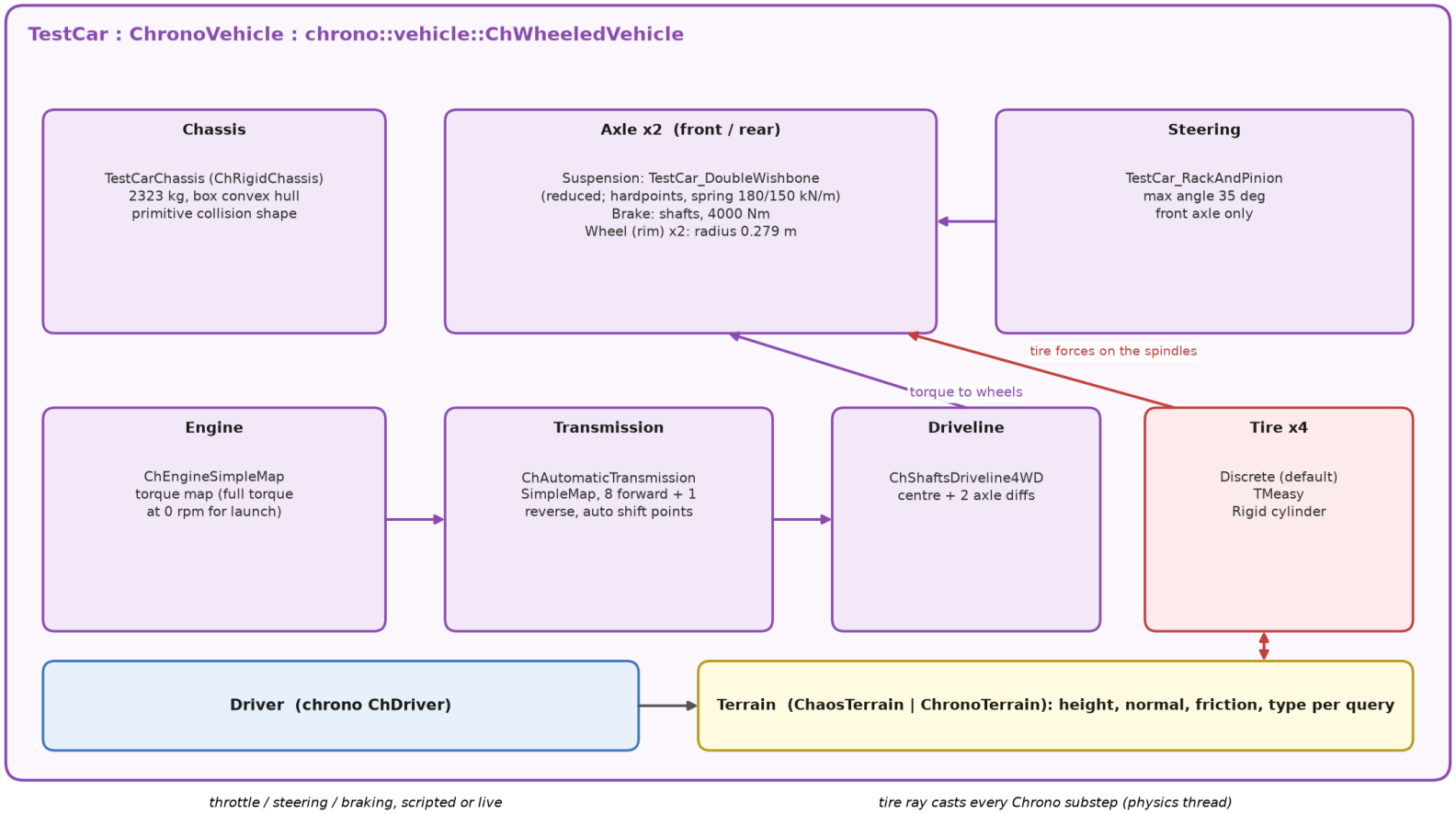


Figure 4: structure of TestCar. All components are Chrono parts; no mesh or data files are loaded.

Component	Value
Chassis	2322.8 kg, inertia (800, 2400, 3000) kg m², centre of mass (0.5, 0, 0.483) m, one box convex hull (x −2.05…2.35, y ±0.74, z 0.48…1.45 m)
Axles	front at x = +1.535 m, rear at x = −1.35 m; wheelbase 2.885 m; spindle ±0.91 m
Suspension	reduced double wishbone on both axles, hardpoints in cm from the axle; front spring 180 kN/m, damper 18 kN s/m; rear 150 kN/m, 15 kN s/m; rest length 0.49 m
Wheels, brakes	rim radius 0.279 m, width 0.241 m, 12.4 kg; shaft brakes, 4000 N m max
Steering	rack and pinion, maximum angle 35°
Powertrain	simple engine map (max 6700 rpm, peak 624 N m at 2500 to 3000 rpm; full torque at 0 rpm so the car can launch, because the simple map has no clutch or torque converter); automatic transmission with 8 forward gears (ratios 0.157 … 1.4925) and reverse (−0.15); 4WD shaft driveline, conical ratio 1/3.31

Tire models (selected with `TireModel` on the vehicle)

Model	How it works	Needs from the terrain	Deforms terrain
Discrete (default)	ring of 120 tread elements × 5 strips; ground sampled at 3 columns × 15 stations, which envelopes obstacles; brush-type friction on hard surfaces, Bekker / Wong / Janosi–Hanamoto terramechanics on soft ones	height, normal, friction, surface type	yes (publishes its ground heightfield)
TMeasy	Chrono ChTMeasyTire, parameters guessed from the tire size and load	height, normal, friction	no
Rigid	ChRigidTire: a cylinder collision shape (radius 0.38 m, width 0.241 m, 33 kg, μ 0.9, restitution 0.1) that collides through the collision backend itself	nothing (it collides)	no

5 The Chaos collision backend

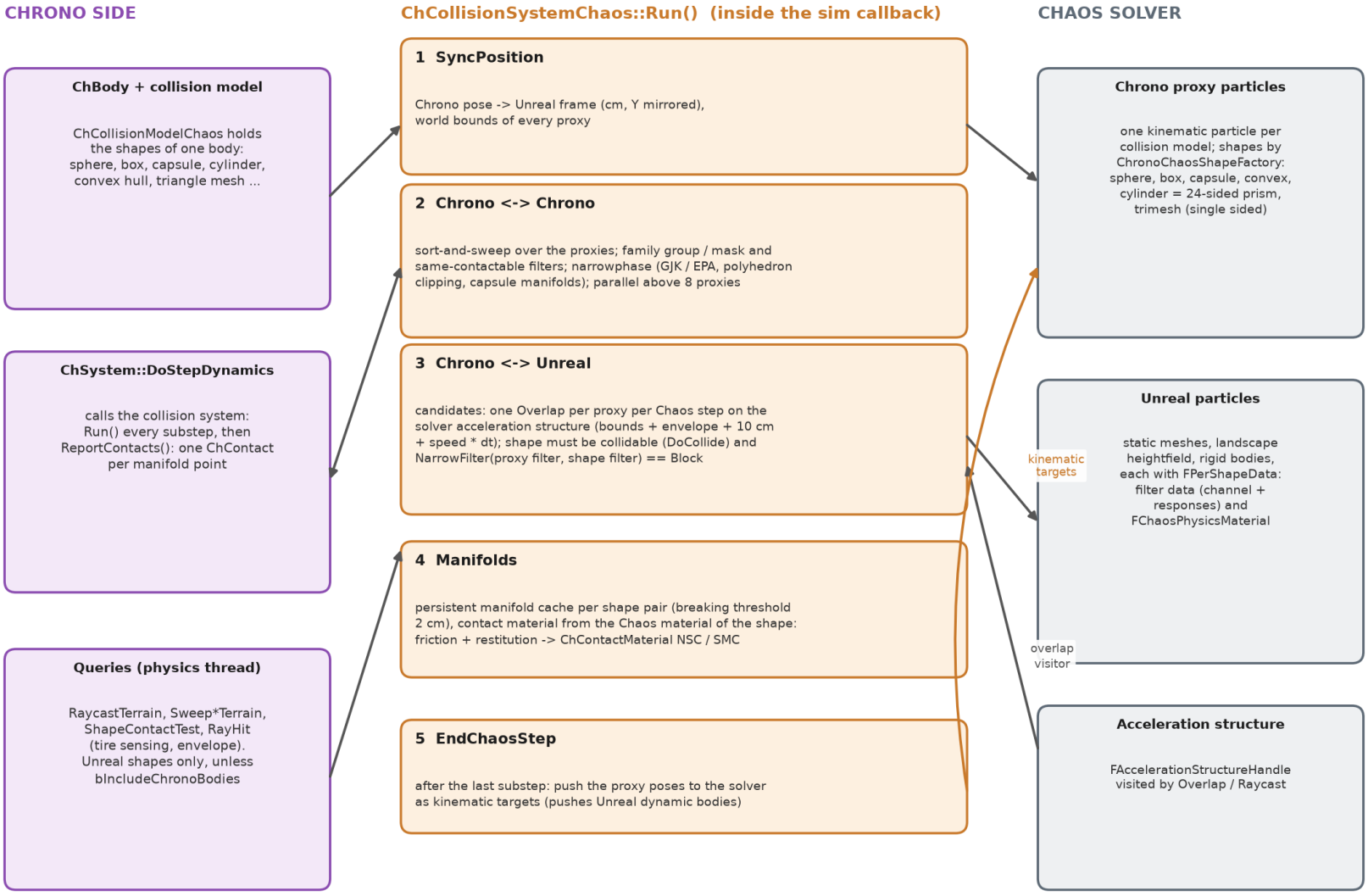


Figure 5: what ChCollisionSystemChaos does each substep. It replaces Chrono's collision system; Chrono keeps its solver, integrators, contact containers and materials.

5.1 Idea

A Chrono system whose collision system is `ChCollisionSystemChaos` is inert until it is bound to a Chaos solver. Once bound, every Chrono collision model is mirrored as a **kinematic proxy particle** in that solver, exactly like one of Unreal's own kinematic bodies. Chrono bodies then collide with each other and with everything else in the Chaos scene (static meshes, landscape heightfields, other rigid bodies), and ray / sweep / overlap queries see the same world as Unreal does.

5.2 Per substep

- **Chrono ↔ Chrono:** exact sort-and-sweep over the proxy list, family group / mask and same-contactable filters, narrowphase with Chaos GJK / EPA (one point for sphere-vs-convex and capsule-vs-capsule), polyhedron face clipping with up to 4 planar points for box / convex pairs, capsule manifolds against faces.
- **Chrono ↔ Unreal:** once per Chaos step one `0verlap` of the solver's acceleration structure per proxy gathers candidate shapes; the bounds are inflated by the envelope + 10 cm + 1.5 × speed × dt, and the list is re-gathered only if the proxy moved more than half of the inflation. A shape is a candidate only if it is sim-enabled, has a valid filter, is a triangle mesh only with complex-as-simple, and the filter of the proxy blocks it (section 7).
- **Manifolds:** persistent per shape pair; points whose separation exceeds the breaking threshold (2 cm) are dropped; the reaction cache warm-starts NSC. Each manifold point becomes one Chrono contact whose second model is a fixed, collision-disabled *world proxy body*, and whose material is built from the Chaos material of the Unreal shape.
- **Queries:** `RaycastTerrain`, `SweepCapsuleTerrain`, `SweepSphereTerrain`, `ShapeContactTest` and `RayHit` run on the physics thread against the same candidates. Terrain queries ignore the Chrono proxies by default, so a tire cannot hit its own chassis. A hit returns point, normal, distance, friction, restitution and the pointer identity of the physical material.
- **Envelope:** the narrowphase cull distance. Chaos shapes are not inflated, so contact points and ray hits lie on the true surface (Bullet reports points on the envelope-inflated surface).

5.3 Shape coverage (Chrono shape → Chaos geometry)

Chrono shape	Chaos backend	Note
sphere, box, capsule, convex hull, point	supported (FImplicitSphere, TBox, FCapsule, FConvex, tiny sphere)	capsule axis is Z like Chrono
cylinder	convex 24-sided prism	rolling is faceted: see section 10, rolling scene
cone, ellipsoid, rounded box / cylinder	convex hull of a sampling, rounding as Chaos margin	approximations
triangle mesh	single-sided FTriangleMeshImplicitObject	back faces do not collide; mesh ↔ mesh uses the sweep-sphere radius as GJK thickness
cylinder shell, barrel, segment / triangle (FEA), 2D shapes	not supported	skipped with a warning; a model with no supported shape gets no proxy

5.4 Deliberate limits

- Contacts against Unreal *dynamic* bodies are one-way inside Chrono (treated as immovable); Unreal dynamics are pushed by the proxies through Chaos' kinematic pipeline.
- Runs on the physics thread only; stepping a CHAOS system on the game thread is unsupported. The Chrono callback is not rewound and is not part of Unreal's resimulation.
- Game-thread scene queries (`LineTraceByChannel`) do not see the Chrono proxies (they are physics-thread-only particles).
- Per-face physical materials of a mesh are not resolved: the first material of the shape is used.

5.5 The Bullet backend, for comparison

- Chrono's bundled Bullet (double precision) only sees Chrono bodies. The level is exported on the game thread by `FUnrealCollisionExtractor`: actors tagged `ChronoStatic` (actor and component tag), landscape heightfield patches as triangle meshes, converted to static ChBody shapes (box, sphere, capsule, convex, mesh) with a material built from the physical material.
- `FStaticCollisionGrid` streams the exported bodies in and out of the system in 64 m cells within 250 m of the vehicle (50 m hysteresis, at most 128 attach / detach per step) from a worker thread; the result can be baked to an archive file.
- The tires sense the ground with Bullet ray casts in the `TERRAIN_QUERY_GROUP`. Bullet uses its own thread pool (8 threads by default).

6 Physical materials

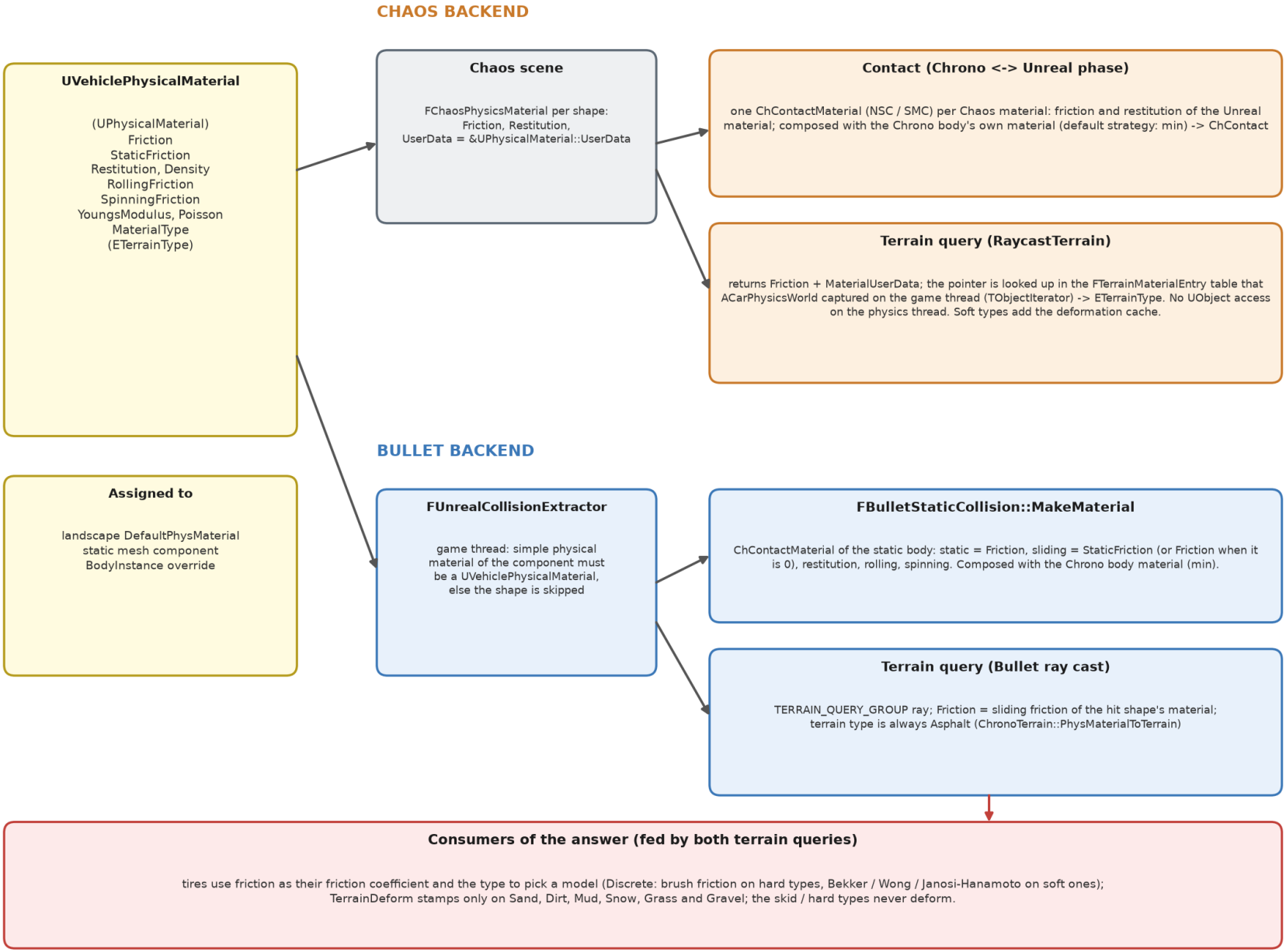


Figure 6: from the authored UVehiclePhysicalMaterial to the answers each backend gives.

6.1 What a UVehiclePhysicalMaterial carries

It derives from `UPhysicalMaterial` and adds the surface type and a few Chrono parameters. It is assigned the usual Unreal way: as the landscape's default physical material or as the physical material override of a static mesh component. Both backends read it, but through different paths and with different fields:

Property	Chaos backend	Bullet backend
Friction	contact friction of the Unreal shape (Chaos material) and the friction of the terrain query	static friction of the exported body; also the sliding friction when StaticFriction is 0 (see 6.3)
StaticFriction	not used by the Chrono contact	sliding friction of the exported body when > 0; this is the value the tire models read as the terrain friction
Restitution	contact restitution	contact restitution
Rolling / Spinning friction	not used	rolling and spinning friction of the exported body
MaterialType (ETerrainType)	resolved from the pointer identity in the terrain query: selects the tire model branch and whether the ground deforms	not used: the Bullet terrain always reports Asphalt
Density, Young's modulus, Poisson	not used	carried by ChColliderInfo; the NSC material built today does not use them

6.2 Contacts and the "min" rule

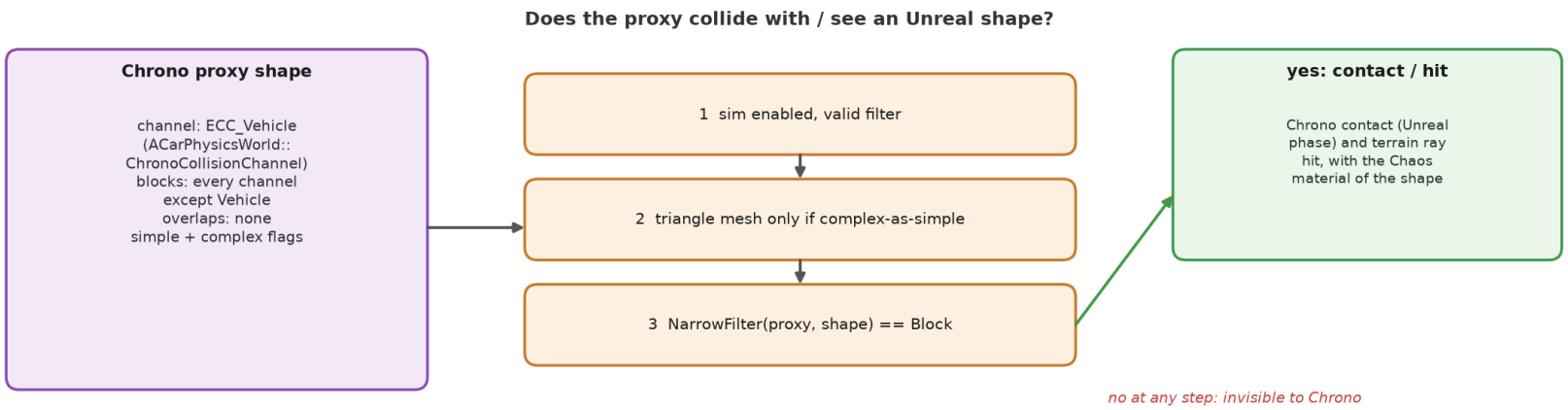
A Chrono contact needs one material per side. For the Unreal side the Chaos backend builds a Chrono contact material (NSC or SMC) from the friction and restitution of the Chaos material of the hit shape, one per distinct Chaos material, cached. Chrono then combines the two sides with the default composition strategy, which is the **minimum** for friction and restitution. Example: tire μ 0.9 on asphalt μ 0.9 gives 0.9; the same tire on sand μ 0.6 gives 0.6.

6.3 How the surface type reaches the physics thread without touching a UObject

When `ACarPhysicsWorld` builds the vehicle setup on the game thread it walks every loaded `UVehiclePhysicalMaterial` and stores `{&UPhysicalMaterial::UserData, MaterialType, friction values}` in the setup. Chaos attaches exactly that address to its own material as `UserData`, so a terrain ray cast hands back a pointer that the physics thread compares against the table; it never dereferences it. An unknown pointer resolves to Asphalt.

The tires use the friction as their friction coefficient and the type to choose the model. On **Bullet** the terrain friction is the *sliding* friction of the hit shape, which was taken from `UPhysicalMaterial::StaticFriction`. Unreal leaves that at 0 unless it is authored, so the Discrete and TMeasy tires had no traction on Bullet until `FBulletStaticCollision::MakeMaterial` was changed to fall back to `Friction` (found by the comparison test, section 10).

7 Collision channels and collision families



What that means for Unreal objects

Unreal object	collision setup	collides with Chrono / sensed by tire rays
Landscape, static meshes (BlockAll)	blocks every channel	yes
Object blocking WorldStatic only, or ignoring Vehicle	response to Vehicle = Ignore / Overlap	no
Visual vehicle meshes (APhysicsVehicle)	NoCollision	no (no self collision, no self sensing)
Unreal rigid bodies (simulating)	blocks Vehicle	proxy pushes them (kinematic); Chrono sees them as immovable
Another Chrono body	not an Unreal shape	Chrono collision families (group / mask), not channels

Bullet backend: Chrono collision families (ChronoCarPhysics/Tools/Common.h)

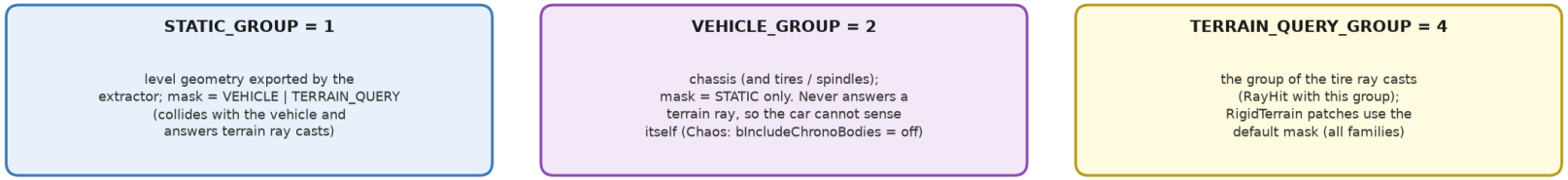


Figure 7: which Unreal shapes Chrono proxies collide with and sense, and the Bullet collision families.

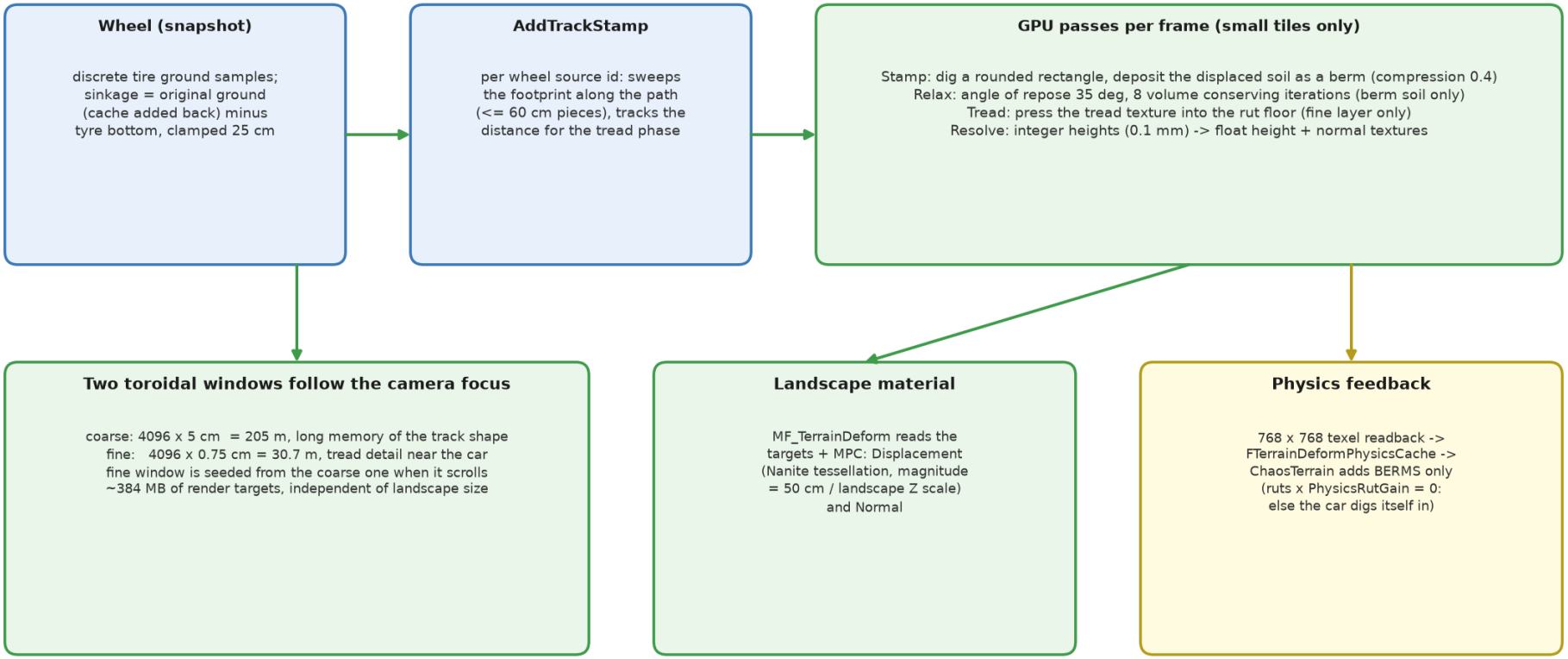
7.1 Chaos backend: Unreal collision channels

- Every proxy shape belongs to the collision channel configured on the physics world: `ChronoCollisionChannel`, default **ECC_Vehicle**. It **blocks every channel except its own** (`BlockChannels = ~0 & ~(1 << Channel)`), overlaps none, and takes part in both simple and complex collision.
- An Unreal shape is therefore seen by Chrono only if its collision responds with *Block* to the Vehicle channel. Landscapes and static mesh actors with the default BlockAll profile do; anything that ignores or only overlaps Vehicle is invisible to the car, for contacts and for tire rays alike.
- The visual vehicle meshes have *NoCollision*, so there is no self collision and no self sensing. Tire ray casts additionally skip the Chrono proxies (`bIncludeChronoBodies = false`).
- The decision is evaluated by Chaos' own rules: `DoCollide` (sim-enabled, valid filter, triangle meshes only with complex-as-simple) and `NarrowFilter(proxy filter, shape filter) == Block`, in the candidate gather and in every scene query.
- `bCollideWithUnrealGeometry` turns the whole Unreal phase off, which gives a pure Chrono scene (used by the rigid body comparison scenes). `bProxySimEnabled` controls whether the proxies push Unreal rigid bodies.

7.2 Bullet backend: Chrono collision families

- Bullet does not use Unreal channels. Exported level geometry belongs to `STATIC_GROUP` (1) with mask `VEHICLE | TERRAIN_QUERY` (6); the vehicle is in `VEHICLE_GROUP` (2) with mask `STATIC` (1).
- Tire ray casts use `TERRAIN_QUERY_GROUP` (4): only bodies whose mask contains that bit answer, which the vehicle never does, so it cannot sense itself. Static bodies never collide with each other (`DisallowCollisionsWith(STATIC_GROUP)`).
- Whether a piece of level geometry reaches Bullet at all is decided by tags, not by channels: the actor and its component both need the `ChronoStatic` tag, and the component needs a UVehiclePhysicalMaterial.

8 Terrain deformation



Only Sand, Dirt, Mud, Snow, Grass and Gravel surfaces deform (decided by the physical material type). Deformation is Chaos-backend only: the Bullet terrain query does not read the cache. Landscape components use Shadow Cache Invalidation = Always so virtual shadow maps do not keep stale shadows.

Figure 8: GPU deformation of the Nanite landscape and its feedback to the physics.

9 Backend comparison test: method

The level `/Game/Maps/BackendComparison` contains an `AChronoBackendComparison` actor that runs the same scenarios on the Bullet and on the Chaos backend and compares the outputs. It is built to make a difference attributable to the backend and nothing else:

- **Identical inputs.** The vehicle is driven by a script of throttle, steering and braking keys (settle 1.5 s, accelerate, steer left then right, lift, brake) that is a function of simulation time and is applied on the physics thread, so the frame rate cannot influence the result. The rigid body scenes are generated from a fixed seed and built on the physics thread as plain Chrono bodies; the Unreal level is excluded, so both backends simulate the identical Chrono system.
- **Identical scene.** The vehicle drives over a slab, a 1.5° ramp and two 8 cm bumps. Chaos uses them directly; Bullet gets them through the usual static collision export. The ground carries an asphalt UVehiclePhysicalMaterial.
- **Recording.** The state of every body (position, orientation, linear and angular velocity) is recorded at every Chrono substep (360 Hz; 120 Hz for the rigid scenes) on the physics thread; the two runs are compared on the same time grid. Telemetry (speed, engine rpm, gear) is recorded too.
- **Noise floor.** Every scenario is run twice per backend. How far a backend differs from itself is the smallest difference that can be attributed to a backend.
- **Terrain probe.** Each vehicle run also queries the backend's terrain query at a fixed grid of 600 points over flat ground, ramp and bumps and compares height, normal, friction and surface type.

Scenarios and pass criteria

Scenario	What it exercises	Judged by (all limits are configurable in FBackendComparisonCriteria)
Vehicle: Discrete, TMeasy, Rigid tire	the whole vehicle on both backends, 9.5 s	chassis and wheel position ≤ 5 cm + 2% of the distance driven; chassis orientation ≤ 3°; speed RMS ≤ 3% of the peak; terrain probe: friction ≤ 0.05, height ≤ 1 cm, normal ≤ 2°, no hit or type mismatch
Rolling cylinders and spheres	5 wheel-sized cylinders and 5 spheres launched at 5 m/s with ω = v/r	distance rolled within 2% between the backends
Sparse drop	36 mixed shapes (box, sphere, capsule, cylinder, convex hull) dropped far apart	any body, any time ≤ 15 cm; kinetic energy RMS ≤ 10% of the peak; no body escapes
Pile	200 mixed bodies falling into a walled container (chaotic)	scene statistics only: kinetic energy RMS ≤ 35% of the peak, centre of mass ≤ 10 cm, 90th percentile height ≤ 15 cm, bodies still moving at the end within 10%

Repeatability rows (the same backend run twice) are reported as information: they are the noise floor, not a pass criterion. All tolerances are my choices, set before looking at the final numbers; the only statistic changed afterwards was the pile height (the maximum was replaced by the 90th percentile, because one perched body makes a maximum meaningless in a chaotic stack).

10 Results

Scenario	Result	Headline numbers (Chaos vs Bullet)
Vehicle, Discrete tire	PASS	chassis position error max 0.21 m over an 82 m drive (0.26%); orientation 0.7°; speed RMS 0.1% of peak; terrain probe identical (height 2e-5 m)
Vehicle, TMeasy tire	PASS	max 1.28 m over 83 m (1.5%); orientation 2.5°; speed RMS 0.2% of peak; first differences at 3.3 s, growing once the steering starts
Vehicle, Rigid tire	FAIL	max 10.4 m over a 55 m drive; Chaos about 10% slower once the throttle is on; 5 cm apart after 0.8 s (cylinder faceting, see below)
Rolling cylinders and spheres	FAIL	spheres identical (1e-12 m); cylinders roll 17.9 m on Chaos against 25.0 m on Bullet (28% less)
Sparse drop	FAIL	35 of 36 bodies within cm to dm; one capsule gains energy on Bullet and ends 24 m away (error 24.4 m)
Pile	FAIL (noise limited)	energy RMS 1.6% of peak, centre of mass 3.8 cm, P90 height 3.4 cm all agree; Chaos keeps 12% more bodies creeping at the end (limit 10%)

Overall verdict at the default tolerances: **FAIL**. The vehicle with the two force-element tires (Discrete, TMeasy) agrees between the backends; the paths where the collision backend itself generates the contacts (cylinders, capsules) show real, explainable differences.

10.1 The vehicle, three tire models

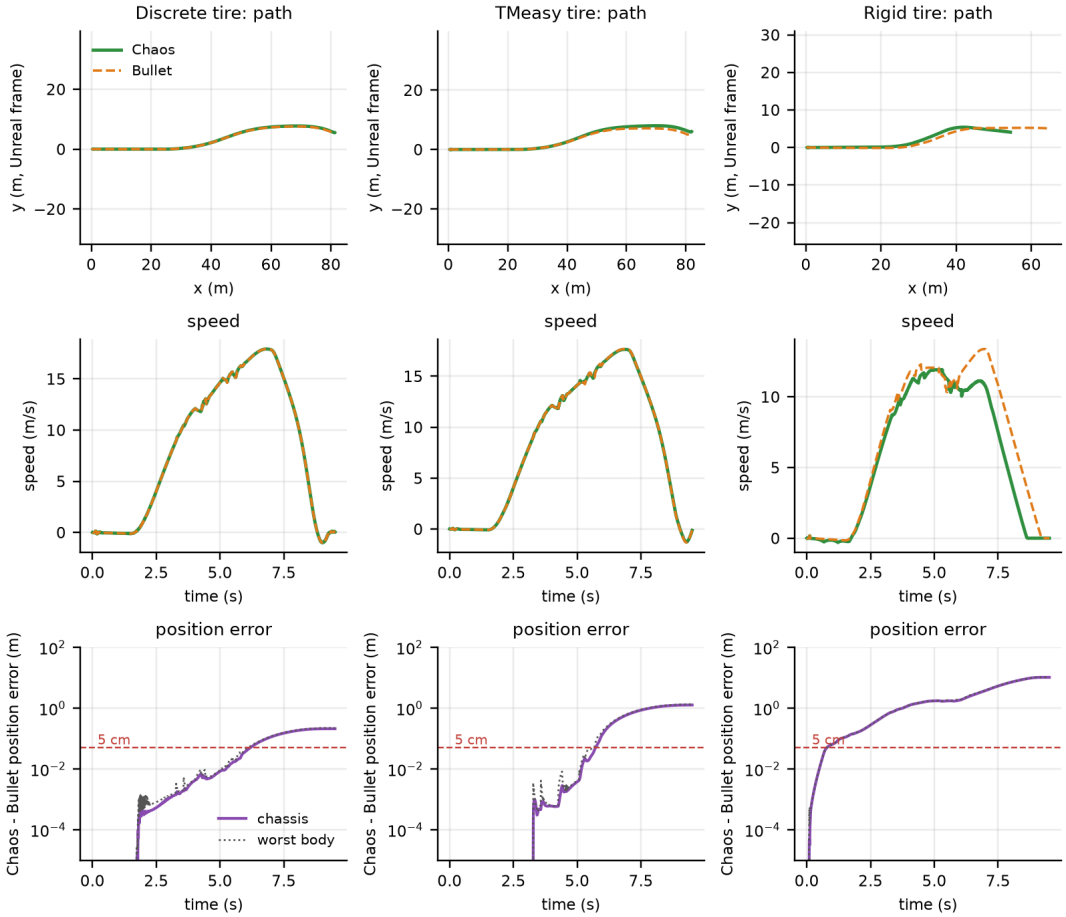


Figure 9: top view of the path, speed, and the position error over time (log scale, 5 cm marked). The Discrete and TMeasy errors are zero until the first differences appear (about 1.9 s and 3.3 s), stay below 1 cm until the steering starts at about 4.5 s, then grow slowly; the Rigid tire already diverges while the car settles on its suspension (5 cm at 0.8 s). Both backends reproduce themselves exactly (0 m between repeats) for the vehicle.

10.2 The bug the test found: Bullet friction

The first runs showed the Bullet car barely moving (engine at 6000 rpm, 0.2 m/s). The terrain probe made the cause visible: for the same ground the Chaos query returned friction 0.9 and the Bullet query 0. The Bullet material took its sliding friction from `UPhysicalMaterial::StaticFriction` (0 unless authored) and the force-element tires read that as their friction coefficient. The fix is in `FBulletStaticCollision::MakeMaterial`: sliding friction falls back to `Friction` when no static friction is authored. Authored values behave as before. The Rigid tire is unaffected because the contact solver uses the static friction.

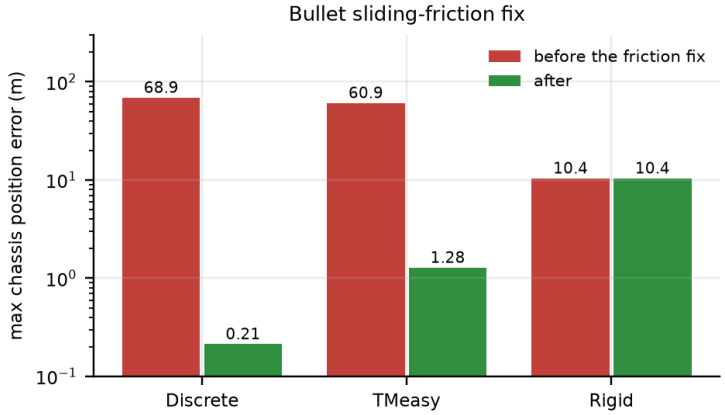


Figure 10: maximum chassis position error before and after the fix.

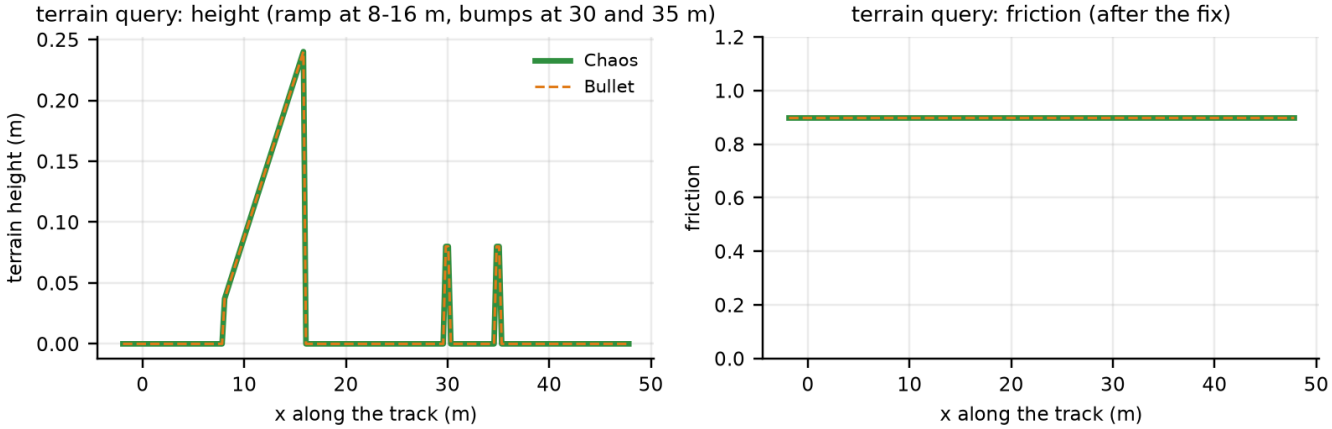


Figure 11: the terrain probe along the track (ramp 8 to 16 m, bumps at 30 and 35 m): the two backends return the same height (largest difference 2e-5 m), normal, friction and surface type at all 600 points.

10.3 Rolling cylinders: the faceting of the Chaos cylinder

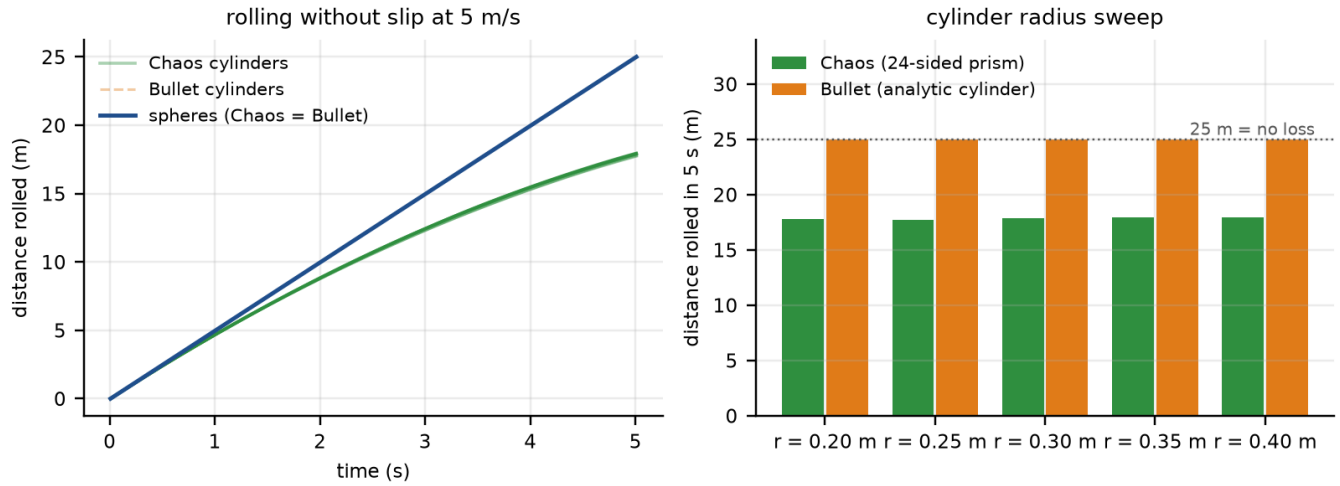


Figure 12: cylinders and spheres launched at 5 m/s and rolling without slip. Spheres match to 1e-12 m. Chaos approximates a cylinder with a 24-sided convex prism, every facet is a small inelastic impact, and the cylinders roll 28% less distance in 5 s than on Bullet's analytic cylinder (which loses none). As a control, raising the prism to 96 sides (a one-constant change, since reverted) cut the difference to 11%, which confirms the cause. This is also why the Rigid tire (a cylinder) differs between the backends.

10.4 Sparse drop and pile

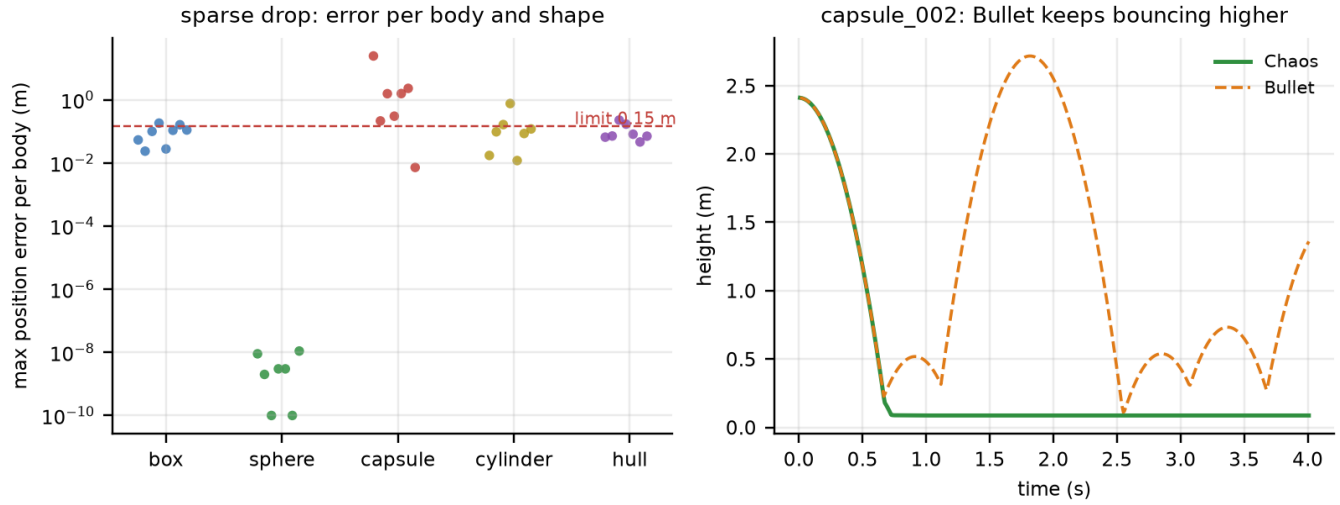


Figure 13: left, the largest position error of every body by shape (spheres 1e-8 m, boxes up to 18 cm, convex hulls 23 cm, cylinders 0.76 m, capsules up to 24 m). Right, the capsule that differs: after landing it keeps bouncing on Bullet with growing speed (3 to 14 m/s), i.e. energy is injected by Bullet's capsule against box contact, while Chaos settles it normally.

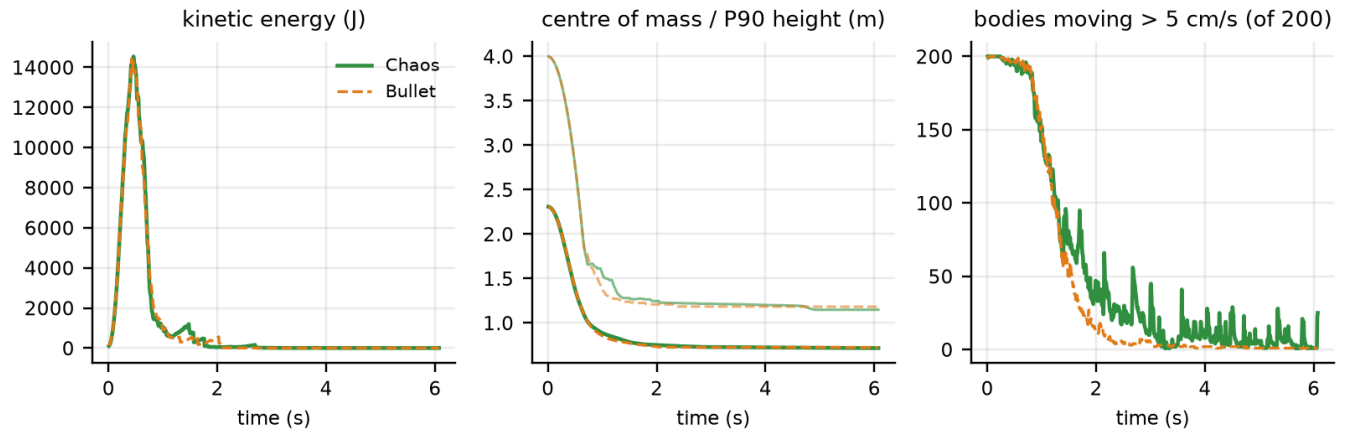


Figure 14: the pile, scene level quantities of both backends. Individual trajectories diverge chaotically (5 cm after 0.07 s) as they would for any two solvers that differ in the last bit; the statistics agree. The count of bodies still moving at the end differs, because the Chaos run keeps a few bodies creeping.

10.5 Repeatability and limits of this evidence

- The vehicle runs repeat bit for bit on both backends, and are identical between a windowed run and a headless `-nullrhi` run.
- In the Chrono-only rigid scenes the **Chaos backend is not bit-repeatable**: it differs from itself by 0.04 to 0.7 m in the sparse drop and 2 to 3 m in the pile (Bullet differed from itself twice as well: 7 mm in the rolling scene, 4.6 m in one pile run). Running the Chaos narrowphase serially did not remove it, so it is not the parallel for; the cause was not tracked down. The pile verdict is therefore noise limited and flips between runs.
- Everything was measured on one machine, one engine build, 120 Hz async physics, NSC contacts, 140 solver iterations. The test covers flat ground, one ramp and two small bumps, not a landscape heightfield with Bullet, and not SMC contacts.
- The tolerances are explicit engineering judgements, not standards. A FAIL means the difference exceeds them; the report gives the numbers so the reader can judge.

10.6 Reproducing

```
UnrealEditor.exe UnrealChrono.uproject /Game/Maps/BackendComparison -game -nullrhi -nosound -ChronoCompareQuit
```

The exit code is 0 when every required check passed and 1 otherwise. The report (`report.md`, `summary.json`) and the CSVs (raw trajectories, error over time, terrain probe, aggregate series) go to `Saved/BackendComparison/<timestamp>/`. Quick-run switches: `-ChronoCompareVehicleOnly`, `-ChronoCompareRigidOnly`, `-ChronoCompareNoRepeat`, `-ChronoCompareTire=D|T|R|DTR`, `-ChronoCompareOnly=<name part>`, `-ChronoCompareSerialChaos`.

Appendix: where things are

Path	Content
Plugins/ChronoCore	Chrono core, Bullet backend, FChronoSimulation, Chaos backend (Private/ChaosCollision), FChronoConversions
Plugins/ChronoVehicle	chrono_vehicle sources
Source/UnrealChrono/ChronoCarPhysics	FChronoVehicleWorld, TestCar, tires, ChaosTerrain / ChronoTerrain, Driver
Source/UnrealChrono/UECarPhysics	ACarPhysicsWorld, APhysicsVehicle
Source/UnrealChrono/Tools	Bullet export: UnrealCollisionExtractor, StaticCollisionGrid; FVehicleSnapshot
Source/UnrealChrono/TerrainDeform, Shaders/TerrainDeform	deformation subsystem, render passes, compute shaders
Source/UnrealChrono/Benchmark	AChronoBackendComparison, rigid body scenes, analysis
Scripts/	create_test_level.py, create_comparison_level.py